Ask the ERP

Developer Handoff Specification

Natural-language search and action assistant for the school ERP — schema, entity-resolution strategy, API contract, permission rules, and a test plan, packaged for a developer to build the production backend and frontend and take it to live deployment.

Document	Ask the ERP — Developer Specification
Version	1.0
Date	August 12, 2026
Status	Ready for implementation — no production backend exists yet (see §1.1)
Audience	Backend engineer, frontend engineer, ML/NLP engineer (optional tier)

1. Overview

Ask the ERP lets a school-office user type a request in plain English — a search, a payment, a status change, a concession or cancellation request, a support ticket, a bus route change, or a report — instead of navigating separate screens for each. The system parses the sentence into a structured intent, resolves it against real records (with disambiguation when the input is ambiguous), shows the user an explicit confirmation of what is about to happen, and only then executes it.

1.1 What exists today vs. what this spec asks for

IMPORTANT — READ FIRST

A fully interactive, client-side **design preview** of this feature exists and is linked in §11. It is a static HTML/JS mock: all data is hardcoded, all matching runs in the browser, and nothing is persisted or connects to a real database. It is the UX reference, not the implementation.

No production backend, database schema, trained model, or React integration exists yet. Everything in this document — the API contract, data models, and matching strategy — is a specification to build against, not a description of running infrastructure.

The preview's entity-resolution logic (fuzzy string matching, §3) is production-quality enough to reuse or port directly; its intent detection (regex/keyword rules, §3.5) is not — that part exists only to make the UI demo-able and should be replaced per §3.5.

1.2 Goals

- Understand loosely-phrased, typo-tolerant natural-language requests without requiring exact keywords.
- Resolve a request to a specific record (or the right small set of candidates) before anything happens.
- Never execute a financial, status, or data-altering action without an explicit user confirmation step.
- Route sensitive requests (concessions, cancellations, status changes) through the correct approver.
- Fail safely: an unrecognized or ambiguous request should ask the user to clarify, never guess.

1.3 Non-goals

- Free-form chat / multi-turn conversation — each query is parsed and resolved independently.
- Voice input, multi-language support, or handwriting — out of scope for v1.
- Bulk/batch actions (e.g. "mark all inactive students as graduated") — v1 is single-record only.

2. System Architecture

Five stages, each independently testable. Stages 1-2 are the "understanding" layer; stages 3-5 are standard CRUD/workflow that already exists elsewhere in the ERP and should be reused, not rebuilt.

Stage	What it does
1. Query Input	User types a sentence in the search bar. Sent verbatim to the backend — no client-side parsing.
2. Intent Parser	Classifies the sentence into one of 9 actions and extracts slots (name, class, amount, reason, ...). See §4.
3. Entity Resolver	Looks up the extracted slots against real records (students, staff, branches...). Returns 0, 1, or N candidates.
4. Disambiguation / Confirm	0 candidates → "no match" message. 2+ → candidate picker. Exactly 1 → confirmation card showing the exact action before it runs.
5. Action Executor	Only runs after explicit user confirmation. Delegates to the same service/permission layer as the existing UI screens (Make Payment, Students, etc.) — this feature is a new entry point, not a new execution path.

Flow: Query Input → Intent Parser → Entity Resolver → (Disambiguation if needed) → Confirmation → Action Executor → Response rendered back to the same search result area.

3. Matching & NLP Strategy

3.1 Correcting a naming mix-up

"Dijkstra's algorithm" does not apply here

Dijkstra's algorithm finds the shortest path between nodes in a weighted graph — the classic example is turn-by-turn driving directions. It has no notion of strings, words, or similarity and cannot be used to match a mistyped word to the nearest real one.

The correct technique for "user typed something close but not exact, find the nearest valid option" is **fuzzy string matching**, specifically **edit-distance matching** (Levenshtein / Damerau-Levenshtein distance). That is the algorithm specified below and implemented in the reference prototype (§11).

3.2 Two-tier design

Production accuracy needs two cooperating layers. Conflating them is the most common mistake in systems like this — a fuzzy string matcher is not an intent parser, and an intent parser should not be doing entity lookups itself.

Tier	Job	Technique
Tier 1 — Intent Parser	Given a full sentence, decide *which of the 9 actions* the user means and pull out the raw slots (name, amount, reason, page, ...) as text spans.	Recommended: a small trained sequence model (intent classification + slot extraction) or an LLM function-calling call against the schema in §4. Not string-distance based — this is a language-understanding problem, not a lookup problem.
Tier 2 — Entity Resolver	Given a raw slot value (e.g. the text "Ravi Kumr"), match it against the finite, known set of real values (actual student names, the 4 branch names, the 6 fee types, the 15 ticket page names, the 10 bus routes) and return the nearest real one.	Fuzzy edit-distance matching, specified in full in §3.3. This is a lookup problem against a closed vocabulary, which is exactly what edit distance is good at.

The reference prototype (§11) implements Tier 2 in full and stands in for Tier 1 with hand-written keyword rules, which is sufficient to demo the UI but not to ship — see §3.5 for what changes for production.

3.3 Tier 2 algorithm specification

This is implemented and unit-tested in the reference prototype; port it directly rather than re-deriving it.

Step 1 — Distance function: Damerau-Levenshtein (transposition-aware)

Use Damerau-Levenshtein distance, not plain Levenshtein. The single most common real typo is two adjacent letters swapped — "cancle" for "cancel", "reciept" for "receipt". Plain Levenshtein scores a transposition as 2 edits (two substitutions); Damerau-Levenshtein scores it as the 1 edit a human actually made. Using plain Levenshtein forces a wider distance budget to catch these typos, which in turn causes false matches between unrelated words (see the worked example below).

```
function editDistance(a, b):
    # standard DP matrix, plus one extra rule:
    # if a[i] == b[j-1] and a[i-1] == b[j]: allow a transposition
    # at cost 1, in addition to the normal insert/delete/substitute.
    ...
    return d[len(a)][len(b)]
```

Step 2 — Per-word distance budget, scaled by word length

Word length	Allowed edits	Why
≤ 2 characters	0 (exact only)	Words this short ("to", "as", "or") collide with almost anything at distance 1 — never fuzz them.
3–7 characters	1 edit	Covers the overwhelming majority of real typos (one dropped/added/swapped letter) without drifting into unrelated words.
8+ characters	2 edits	Long words ("consolidated", "registration") can safely absorb 2 edits since that is still a small fraction of the word.

Worked example — why the budget must stay tight

With a 2-edit budget applied to 6-7 letter words, `"report"` and `"record"` are only 2 substitutions apart, and `"parent"` and `"payment"` are only 2 edits apart. A query like *"fee report for Ravi Kumar"* would then wrongly fire the *record_payment* intent (because "report" ~ "record"), fail on a missing amount, and return "didn't understand" instead of the fee report the user actually asked for.

This was caught by testing against real typo'd input, not by inspection — build the test set in §9 before tuning thresholds, and re-run it on every change to the distance budget.

Step 3 — Two matching modes

Mode	Used for	How
Single-token fuzzy match	Verbs and single-word keywords: "cancel", "reopen", "pending", "consolidated"...	Tokenize the query; a keyword matches if any token is within its distance budget of it.
Sliding-window nearest match	Multi-word closed vocabularies: student/staff names ("Ravi Kumar"), branches ("RC Puram"), ticket pages, bus routes.	Try an exact substring hit first (free, zero false positives). If none, slide a window of query tokens the same width as each candidate across the query, score each window by normalized edit distance against the candidate, and return the closest one under threshold (prototype uses 0.32 — see calibration note below).

Step 4 — Always try exact match first

Fuzzy matching is a fallback, not a replacement. An exact substring/equality hit costs nothing and has zero false-positive risk, so it must always run first; only fall back to edit-distance scoring when no exact hit exists. This is what keeps the common case (correctly-typed input) both fast and perfectly accurate.

3.4 Calibration guidance

- The 0.32 normalized-distance threshold for the sliding-window matcher and the per-length edit budgets above were tuned against the test set in §9, not derived analytically — re-tune them against your own production query logs once real traffic exists.
- When the entity vocabulary grows (more branches, more staff), re-run the collision check: for every pair of keywords/entity names the system must tell apart, confirm their edit distance from each other exceeds the sum of their individual budgets. If two real options are closer to each other than the budget allows, tighten the budget or add a required companion word instead of loosening it.
- Log every query where the fuzzy path (not the exact-match path) was what produced the final match, together with the matched candidate and score. This is the fastest way to catch a bad threshold in production before it causes a wrong action.

3.5 What Tier 1 should be in production

The reference prototype's intent detection is an ordered cascade of keyword-presence checks ("does the sentence contain something close to 'cancel' AND something close to 'receipt'?"). It is adequate for demonstrating the UI on a fixed set of example phrasings, and its two hardening fixes are worth keeping regardless of what replaces it:

- **Fall through, don't hard-fail.** If a candidate intent's required slot (e.g. amount, for a payment) is missing, don't return "not understood" immediately — continue checking the remaining candidate intents. A wrong guess on an early check should never block a correct match further down.
- **Order intents from most to least specific** so a broad check (e.g. any mention of "fee") never pre-empts a narrower, more confident one (e.g. an explicit concession-reopen phrase).

For production accuracy beyond a fixed demo script, replace the keyword cascade with one of:

- **A trained intent-classification + slot-extraction model** (e.g. a small encoder-decoder or a classifier + span extractor) trained on a labeled corpus of real phrasings per action — the standard approach for a fixed, closed action set like this one.
- **An LLM function-calling call** against the exact JSON schema in §4, with the model instructed to return `{understood:false}` rather than invent a slot value that was not actually present in the sentence (this constraint matters — see §4.1).

Either way, Tier 2 (fuzzy entity resolution) stays as the layer directly under it: Tier 1 extracts "the user is asking to cancel a receipt for someone named roughly 'Ravi Kumr'", Tier 2 resolves "Ravi Kumr" against the real student table.

4. Action Schema

Every parsed query reduces to one JSON object with this shape. `action` and (for search) `entity` select which of the 9 capabilities is meant; `target` identifies which record it applies to; `params` carries the values needed to execute it.

```
{
  "understood": boolean,
  "action": "search" | "record_payment" | "update_status" |
            "request_concession_change" | "request_receipt_cancellation" |
            "create_support_ticket" | "change_transport_route",
  "entity": "students"|"fee_records"|"staff"|"expenditure"|
            "pending_approvals"|"class_consolidated_report",  // only if action=="search"
  "target": { "student_name"?: string, "class"?: string, "name"?: string },
  "filters": { ... },   // only if action=="search" — see per-entity filters below
  "params": { ... }     // action-specific payload — see table below
}
```

Action	Trigger examples	target	params / filters	Approval
search (students)	"students in class 6 in RC Puram"	—	filters: name, class, branch	None — read-only
search (fee_records)	"fee report for Ravi Kumar", "class 6 unpaid term fees"	—	filters: student_name, class, branch, fee_type, status, amount_gt	None — read-only
search (staff)	"active teachers in RC Puram"	—	filters: name, branch, status	None — read-only
search (expenditure)	"expenditure over 20000 paid by cash"	—	filters: category, payment_mode, branch, amount_gt, amount_lt	None — read-only
search (pending_approvals)	"what's pending for my approval"	—	filters: branch	None — read-only, scoped to caller's role (§7)
search (class_consolidated_report)	"class consolidated report for RC Puram"	—	filters: branch	None — read-only
record_payment	"record a payment of 5000 term fee for Priya Sharma in class 8"	student_name, class	fee_type, amount	None — executes on confirm (same as Make Payment screen)
update_status	"mark Ravi Kumar as inactive"	name	status: Active|Inactive|Graduated	**Principal** approval required
request_concession_change	"reopen the term fee concession for Priya Sharma because ..."	student_name, class	bucket, reason (required, never inferred)	**Admin Officer** approval required

Action	Trigger examples	target	params / filters	Approval
request_receipt_cancellation	"cancel the receipt for Ravi Kumar"	student_name, class	reason (required, never inferred)	**Admin Officer** approval required
create_support_ticket	"raise a ticket, the print button doesn't respond on Fee Receipts"	—	type: bug|feature|improvement, page, description	None — sent directly to support inbox
change_transport_route	"change the bus route for Ravi Kumar to North Route"	student_name, class	route	None — executes on confirm (same as Bus Routes screen)

4.1 Slot-extraction rules that must hold

- **Never invent a value the sentence didn't state.** Most concretely: `reason` for a concession/cancellation request must be `null` if the user gave no reason — the frontend then shows a required, empty reason field rather than a plausible-sounding fabricated one. This applies to every slot: if it isn't in the sentence, it's null, and the UI asks for it rather than the parser guessing.
- `understood: false` is a valid and expected output, not a failure mode — an ambiguous, off-topic, or gibberish query should return this rather than force a best-effort guess into one of the 9 actions.
- Class/grade is optional almost everywhere; it exists only to disambiguate same-named students, never as a required filter on its own.

5. Data Models

These mirror the fields the parser and resolver need; they are the projection this feature reads, not a full schema — join against the existing ERP tables for anything not listed.

Student

Field	Type	Notes
`id`	`string`	Primary key used once resolved — never trust a client-supplied id without re-checking it matches the resolved name/class.
`name`	`string`	Not unique — duplicate names across branches/classes are expected (see disambiguation, §7).
`admission_no`	`string`	Shown in the disambiguation candidate list.
`class`	`string`	"5"–"12", or LKG/UKG/Nursery.
`branch`	`string`	One of the school's fixed branch names.
`status`	`enum`	Active | Inactive | Graduated.
`term_balance`, `admission_balance`, `transport_balance`, `hostel_balance`	`number`	Per fee-type outstanding balance, in the base currency's smallest display unit (rupees, not paise).
`concession_locked`	`string[]`	Which fee-type buckets currently have their concession locked (candidates for request_concession_change).
`transport_route`	`string | null`	Null/empty = not currently on transport; change_transport_route must reject a null-route student with an explanatory message, not silently no-op.

Receipt

Field	Type	Notes
`receipt_no`	`string`	e.g. RC-2026-0142.
`student_id`	`string`	FK to Student.
`date`	`date`	
`amount`	`number`	
`fee_types`	`string`	Human-readable label, e.g. "Term Fee".
`status`	`enum`	Active | Cancelled — only Active receipts are cancellation candidates.

PendingApproval

Field	Type	Notes
`id`	`string`	

Field	Type	Notes
kind	enum	concession | cancellation | status_change — union of the 3 approval-gated actions.
student_id	string	
detail	string	Human-readable summary of what's being requested.
reason	string	The reason text captured at request time.
requested_by	string	User id of the requester, for audit.
requested_on	date	
branch	string	
approver_role	enum	admin_officer | principal — drives who sees it in their queue (§7).

ClassConsolidatedReportRow

Field	Type	Notes
branch	string	
grade	string	
total_students	number	
paid_fee	number	Sum of collected fees for the class.
balance_fee	number	Sum of outstanding fees for the class.

6. API Contract

Three endpoints, deliberately separated so a read-only understanding step never has side effects, and nothing executes without a step the frontend can put behind a confirm click.

POST /api/ask/interpret — Tier 1 only. No DB access, no side effects.

```
// Request
{ "query": "reopen the term fee concession for Priya Sharma because parent submitted new
income proof" }

// Response
{
  "understood": true,
  "action": "request_concession_change",
  "target": { "student_name": "Priya Sharma", "class": null },
  "params": { "bucket": "term", "reason": "parent submitted new income proof" }
}
```

POST /api/ask/resolve — Tier 2. Read-only; returns candidates, never executes.

```
// Request
{ "action": "request_receipt_cancellation",
  "target": { "student_name": "Ravi Kumr", "class": null } }

// Response — 2 same-name students, so resolution stops here and the
// frontend renders a student picker (not a receipt picker yet)
{
  "status": "disambiguate",
  "level": "student",
  "candidates": [
    { "id": "s1", "name": "Ravi Kumar", "admission_no": "MM2023045", "class": "5",
"branch": "RC Puram" },
    { "id": "s2", "name": "Ravi Kumar", "admission_no": "MM2024118", "class": "8",
"branch": "Kukatpally" }
  ]
}
```

Once a single student is resolved, actions with a second-level candidate set (receipt cancellation, where a student may have several active receipts) return `level: "receipt"` instead, following the same shape.

POST /api/ask/confirm — Executes. Requires an explicit confirmation from a prior resolve.

```
// Request — client echoes back the exact resolved target, never a raw name
{
  "action": "request_receipt_cancellation",
  "student_id": "s1",
  "receipt_no": "RC-2026-0142",
  "params": { "reason": "duplicate entry, payment recorded twice" }
}

// Response
{
  "status": "pending_approval",
  "approval_id": "appr_9F21",
  "message": "Receipt RC-2026-0142 for Ravi Kumar is now awaiting Admin Officer approval."
}
```

Server-side re-validation is mandatory, not optional

The client sends back a resolved `student_id`/`receipt_no` from the prior `/resolve` call — the server must re-verify that id still matches the original free-text target (name/class) and that the calling user's role/branch permits the action, rather than trusting the id blindly. Never let the confirm step execute against an id the server hasn't re-checked — this is the standard IDOR-style risk for any resolve-then-confirm flow.

Search endpoints (§4 filters) should be simple `GET` or `POST` calls into the existing search/reporting services already backing the Students/Fee/Staff/Expenditure/Reports screens — do not duplicate that query logic here.

7. Disambiguation, Confirmation & Permission Rules

7.1 Resolution outcomes

Candidates found	UI behavior
0	"No match" message. Never fabricate a near-miss result silently.
1	Go straight to the confirmation card — no extra click to "select" the obvious match.
2+	Render a candidate picker (name, admission no., class, branch, status per row) and wait for the user to pick one before showing any confirmation.

7.2 Confirmation is mandatory for every write action

record_payment, update_status, request_concession_change, request_receipt_cancellation, create_support_ticket, and change_transport_route must all render an explicit "this is about to happen — confirm or cancel" card stating the exact values that will be used (amount, fee type, student, new status, route, etc.) before calling `/api/ask/confirm`. No action is ever a single click from the search box.

7.3 Approval routing

Action	Executes immediately on confirm?	Approver
search (all entities)	N/A — read-only	—
record_payment	Yes	—
change_transport_route	Yes	—
create_support_ticket	Yes (goes to support inbox, not a data change)	—
update_status	No — creates a pending request	Principal
request_concession_change	No — creates a pending request	Admin Officer
request_receipt_cancellation	No — creates a pending request	Admin Officer

The pending_approvals search entity (§4) must be scoped server-side to what the calling user's role is actually allowed to approve — an Admin Officer's "what's pending for my approval" query should never return status-change requests awaiting the Principal, and vice versa.

7.4 Audit logging

- Log the raw query, the parsed intent, the resolved target, and the final action for every confirmed execution — this is the paper trail for "who changed what and why" and should reuse whatever audit log the existing Make Payment / Students screens already write to.
- Store the `reason` text verbatim on concession/cancellation/status requests; it's the primary field an approver reviews.

8. Frontend UX Flow

One search bar, one result area below it that swaps between five states. The interactive reference prototype (§11) implements all five and is the visual source of truth — this section is the state list, not a redesign.

State	Trigger	Content
Loading	Query submitted	Spinner + "Understanding your request…"
Not understood	Tier 1 returns understood:false	Amber notice + a hint to add a class/branch/name/fee type to narrow it down
Result table	action="search"	Columns per entity (§4), empty-state row when 0 results
Candidate picker	2+ resolved candidates	List of tappable rows; picking one re-runs resolution
Confirmation card	Exactly 1 candidate, write action	Exact action description, reason field if required, Confirm/Cancel buttons
Success / result	After confirm	Green confirmation with the concrete outcome (receipt no., new status, approval routing, etc.)

- Example chips under the search bar seed a few representative queries per action — keep these in sync with the test set in §9 so the shipped examples are always ones that are known to work.
- The Confirm button on a reason-requiring action must stay disabled until the reason field is non-empty — never let an empty reason reach the confirm endpoint.

9. Test Plan / Acceptance Criteria

Minimum bar before shipping: every row below passes against the real Tier 1 + Tier 2 implementation, run as an automated regression suite (the prototype's Playwright-driven test harness is a reasonable template). Re-run the full set after any change to distance budgets or keyword lists.

Query	Expected
students in class 6 with unpaid term fees	search / fee_records, class=6, status=unpaid
fee report for Ravi Kumar	search / fee_records — 2 results (same-name students), no false disambiguation
expenditure over 20000 paid by cash	search / expenditure, amount_gt=20000, mode=cash
record a payment of 5000 term fee for Priya Sharma in class 8	record_payment, amount=5000, fee_type=term
mark Ravi Kumar as inactive	update_status → disambiguation (2 candidates) → Principal approval
reopen the term fee concession for Priya Sharma because parent submitted new income proof	request_concession_change, bucket=term, reason captured verbatim
cancel the receipt for Ravi Kumar	request_receipt_cancellation → student disambiguation → receipt disambiguation (2 receipts on s1)
what's pending for my approval	search / pending_approvals, scoped to caller's role
show the class consolidated report for RC Puram	search / class_consolidated_report, branch filter applied
change the bus route for Ravi Kumar to North Route	change_transport_route → disambiguation → reject with "not on transport" for the student without a route
raise a ticket, the print button doesn't respond on the Fee Receipts page	create_support_ticket, type=bug, page="Fee Receipts", description extracted
cancle the reciept for Priya Sharma *(typo)*	Same result as the correctly-spelled query — Tier 2 must absorb this
reopn the term concesion for Priya Sharma becase ... *(typo)*	Same result as the correctly-spelled concession-reopen query
show class consolidatd report for Kukatpaly *(typo)*	Same result with branch correctly resolved to Kukatpally
asdkj not a real query	understood: false — must never force a guess
fee report for Ravi Kumr *(typo, ambiguous name)*	Regression check: a fuzzy-matched name in a *search* query returns the filtered list directly — it must not be misrouted into a single-record action and then fail on a missing slot (see §3.3 worked example)

10. Non-Functional Requirements & Deployment

10.1 Performance

- Tier 1 + Tier 2 combined should return in well under 500ms for the interpret/resolve calls — these block the search UI's loading spinner.
- Fuzzy matching over the sliding-window candidates is O(candidates × query-tokens × word-length²); for the entity sizes here (students, staff, a handful of branches/routes/pages) this is trivial, but if the student table grows very large, pre-filter by any exact tokens present (e.g. a class number) before running the fuzzy pass over names.

10.2 Security

- Standard input handling for the free-text query: parameterize all DB access, never string-concat the raw query into a SQL/search query.
- Re-validate every resolved id against the caller's role/branch on confirm (§6) — the resolve step is not a trust boundary.
- Rate-limit the interpret/resolve endpoints per user to prevent enumeration of student names via repeated fuzzy queries.

10.3 Observability

- Log every query with: raw text, parsed intent (or understood:false), which tier resolved it (exact vs. fuzzy), resolved candidate count, and final action taken.
- Track a weekly "understood:false" rate and a "fuzzy-tier resolution" rate — a rising trend on either is the earliest signal that Tier 1 needs retraining or Tier 2 needs re-calibration (§3.4).

10.4 Deployment checklist

- Feature-flag the search bar's rollout per branch/role so it can be disabled instantly without a deploy if Tier 1 accuracy is unacceptable in early use.
- Ship the automated regression suite from §9 in CI, gating merges to the parser/resolver code.
- If Tier 1 is an LLM function-calling call: pin the model version, set a strict JSON schema/tool definition matching §4, and define a timeout + fallback to "understood:false" rather than blocking the UI indefinitely.
- If Tier 1 is a trained model: version the model artifact alongside the code, and keep the training corpus/labels in the repo so the model is reproducible.

11. Reference Prototype

A fully interactive, client-side mock implementing everything in §7-8 (all 5 UI states, all 9 actions, both single- and two-level disambiguation) with the Tier 2 fuzzy matcher from §3 implemented exactly as specified — port it directly rather than re-deriving it.

Live preview: https://claude.ai/code/artifact/51f8c9f0-ae7c-4887-b337-2ea15feddeb6

What to reuse vs. rebuild from the prototype

Reuse directly: the fuzzy matching functions (edit distance, tokenizer, distance budget, sliding-window nearest-match) — these are the Tier 2 implementation from §3, in plain JavaScript, already tested against the typo cases in §9.

Reuse as a UX reference, rebuild on a real backend: the five UI states and the confirmation-card copy per action — the layout and wording are final, but the prototype's data is a hardcoded in-browser fixture standing in for the API in §6.

Do not reuse: the prototype's intent-detection keyword cascade (its Tier 1) — treat it as a stand-in that made the UI demo-able, not as production logic. Replace it per §3.5.

End of specification.